Literature Review Guide — Adversarial Robustness for Cloud IDS

Thesis: Adversarial Robustness of Deep Learning-Based Intrusion Detection Systems in Cloud Environments **Reproduced paper:** Holla, Polepalli & Sasikumar — *IEEE Access 2025* **Dataset used:** CSE-CIC-IDS2018 **Last updated:** 2026-08-05

HOW TO USE THIS DOCUMENT

Read every section **IN ORDER**. Do not skip ahead.

For each paper write **3–5 sentences** covering:

1. What problem does this paper solve?
2. How does it solve it (the core idea in plain language)?
3. **WHY DOES IT MATTER FOR OUR THESIS SPECIFICALLY?** (I have given you the starting connection — expand it in your own words)

> **Do NOT guess if you cannot find a paper — message the team lead first. Do NOT rearrange sections — the order tells a story. Follow it exactly.**

SECTION 1 — What is an adversarial attack? (READ THESE FIRST — before anything

---

else)

These are the papers that started the whole field. Every later paper compares itself to these. The paper we reproduce (Holla et al., 2025) directly uses three of these four attacks.

1.1 — FGSM (Fast Gradient Sign Method)

Paper: Goodfellow, Shlens, Szegedy — *"Explaining and Harnessing Adversarial Examples"* (ICLR 2015) **Find at:** arXiv:1412.6572

What it does: The model is trained by computing gradients of the loss with respect to the model **weights**. FGSM asks: what if we compute that gradient with respect to the **input** instead? This tells us which direction to nudge each input feature to make the model most confused. FGSM takes ONE step in that direction: `x_adv = x + epsilon * sign(gradient_of_loss_w.r.t._x)`. One step. Very fast. Not very strong — but often enough to fool models.

Exact numbers from the paper we reproduce: Holla et al. (2025) Table 4: FGSM reduces baseline IDS accuracy from **85% → 60%** (–25pp).

Why it matters for our thesis: FGSM appears in THREE places in our code: (1) Evaluation attack via ART in both CUDA and v5 notebooks (Section 13/14) (2) Adversarial training example generation in the CUDA notebook (Section 16) (3) Conceptual basis for the "single cheap gradient step" idea motivating our gradient-free training mix in v5 Explain FGSM as the "cheapest possible attack" — one extra forward-backward pass per sample.

1.2 — PGD (Projected Gradient Descent) — THE MOST IMPORTANT PAPER IN SECTION 1

Paper: Madry, Makelov, Schmidt, Tsipras, Vladu — *"Towards Deep*

Learning Models Resistant to Adversarial Attacks" (ICLR 2018) **Find at:** arXiv:1706.06083

What it does: PGD is FGSM repeated many times with a random start. Start from a RANDOM point inside the epsilon-ball around the clean input. Take a small gradient step. If you have left the epsilon-ball, project back onto it. Repeat N times. The result is a much stronger adversarial example than single-step FGSM.

More importantly: this paper proposes **adversarial training** as a defense. During training, generate PGD adversarial examples from the CURRENT model, then train the model to classify those examples correctly. The model learns to resist attacks, not just clean data.

Key term to define: "Saddle-point problem" — the attacker maximizes loss (finds worst attack), the defender minimizes it. Both happen at the same time.

Exact numbers from the paper we reproduce: Holla et al. (2025) Table 4: PGD reduces baseline accuracy from **85%** → **55%** (–30pp). After defense, accuracy restored to **88%** against PGD.

Why it matters for our thesis: THIS IS THE MOST IMPORTANT PAPER IN OUR LITERATURE REVIEW. READ IT TWICE. Our defended model's Stage 2 is Madry-style adversarial training.

- CUDA notebook (original paper reproduction): static approach — FGSM+PGD+DeepFool examples generated ONCE before training and mixed with clean data (approximation of Madry)
- v5 notebook (our improvement): TRUE Madry formulation — fresh PGD examples generated EVERY mini-batch from the current model weights The "on-the-fly" approach in v5 is more correct and is a genuine methodological improvement over the paper we reproduce.

1.3 — DeepFool

Paper: Moosavi-Dezfooli, Fawzi, Frossard — *"DeepFool: A Simple and Accurate Method to Fool Deep Neural Networks"* (CVPR 2016) **Find at:**

arXiv:1511.04599

What it does: Instead of a fixed epsilon budget, DeepFool finds the SMALLEST possible perturbation that causes misclassification. It linearizes the model's decision boundary locally, computes the smallest step to cross it, takes that step, and repeats until fooled. Produces minimal L2-norm adversarial examples.

Critical methodological note we must mention: ART's DeepFool has NO built-in perturbation budget. In our experiments, uncapped DeepFool produced perturbations 60–150× larger than the eps=0.05 budget (measured: mean $L_\infty \approx 3.0$, max ≈ 7.4 vs. budget 0.05). We added `cap_to_eps()` to clip DeepFool to the same L_∞ budget as FGSM/PGD. This is a **methodological correction** over naive ART usage — mention it explicitly.

Exact numbers from the paper we reproduce: Holla et al. (2025) Table 4: DeepFool reduces baseline accuracy from **85%** → **50%** (–35pp, steepest drop). After defense, accuracy restored to **88%**.

Why it matters for our thesis: DeepFool is the third attack in Holla et al. and in both our notebooks. It represents a philosophically different attack: finding the nearest decision boundary vs. maximizing loss within a budget. Contrast these two approaches explicitly.

1.4 — C&W Attack

Paper: Carlini, Wagner — *"Towards Evaluating the Robustness of Neural Networks"* (IEEE S&P 2017) **Find at:** arXiv:1608.04644

What it does: Formulates adversarial example generation as an optimization problem using Adam optimizer. Minimize perturbation distance, subject to causing misclassification. The paper tested C&W against 10 published defenses and defeated 9 of them.

Why it matters for our thesis: We do NOT implement C&W directly. Cite it for two reasons:

1. It established: "resistance to a weak attack ≠ resistance to a

strong attack" — directly relevant to our gradient masking finding (Section 5)
2. Its philosophy of testing defenses with increasingly powerful attacks is exactly what our Table 10 (adaptive PGD sweep in v5 Section 28) does: sweep 20→50→100→200 PGD iterations to see if defense collapses with more attacker compute

---

SECTION 2 — Attacks that do NOT use the model's gradients (black-box attacks)

Write this introduction before any individual paper:

- **White-box attack**: attacker sees ALL model internals — weights, architecture, gradients
- **Black-box attack**: attacker can only QUERY the model (input → observe output, no gradients)

In a real cloud IDS, the attacker is a network adversary with NO access to the IDS model's source code. Black-box is the more realistic threat model. Our thesis tests BOTH — and finds a SURPRISING result: our model resists white-box attacks well but partially fails against a gradient-free black-box attack. See Section 5 for why.

2.1 — Square Attack

Paper: Andriushchenko, Croce, Flammarion, Hein — *"Square Attack: A Query-Efficient Black-Box Adversarial Attack via Random Search"* (ECCV 2020) **Find at:** arXiv:1912.00049

What it does: Randomly selects a subset of input features, perturbs each by +epsilon or -epsilon. Keeps the change if it fools the model, discards it otherwise. The subset size SHRINKS over iterations (from 30% → 5% of features). Requires no gradient — only model label output queries.

Our implementation note (important to mention): ART's built-in SquareAttack only works on IMAGE data and raises ValueError on tabular data. We wrote our own `tabular_square_attack()` from scratch applying the same shrinking-subset logic to 1D network traffic features. Runs on GPU via forward passes only.

Exact numbers — paper's claims vs. our results: Paper Table 5: Baseline Acc=63%, SR=37% → Defense Acc=84%, SR=16% Our v4/v5 run: Baseline Square Attack success rate = **75.3%** (higher than paper's 37% — likely because our adaptive feature selection left more attack surface: ~42 features vs paper's ~25)

Why it matters for our thesis: Square Attack is directly in our evaluation results (Table 5 in both notebooks). It represents the "practical black-box threat" — the most realistic attacker scenario. The comparison of paper numbers (SR=37%) vs. our numbers (SR=75.3%) is itself an original finding.

2.2 — Boundary Attack (basis for our Boundary-Walk) — CRITICALLY IMPORTANT

Paper: Brendel, Rauber, Bethge — *"Decision-Based Adversarial Attacks: Reliable Attacks Against Black-Box Machine Learning Models"* (ICLR 2018) **Find at:** arXiv:1712.04248

What it does: Starts from an already-misclassified point (heavily corrupted input or wrong-class sample). Takes small random steps ALONG the decision boundary — keeping the example misclassified while slowly reducing distance from the original clean input. Requires ONLY the binary output label — not even probability scores. Zero gradient access.

Why this paper matters more than almost any other for OUR specific results: Our `native_boundary_walk()` in v5 Section 21 is directly based on this idea. When we ran it against our defended model:

- **Gradient-based attacks**: defended model accuracy = **92–95%** (looks very robust)

- **Boundary-Walk (gradient-free)**: defended model accuracy = **34.5%**
- **Gap = 59.15 percentage points** (our threshold for "suspicious" = 15pp — this is 4× threshold)
- Held consistently across all 3 seeds (42, 123, 2024)

Critical connection to make explicitly: Boundary Attack works because it NEVER uses model gradients. If a defense works by hiding/distorting gradients (gradient masking), Boundary Attack is completely unaffected — it ignores gradients. This is the direct link to Section 5.1.

2.3 — JSMA / Sparse Attack

Paper: Papernot, McDaniel, Jha, Fredrikson, Celik, Swami — *"The Limitations of Deep Learning in Adversarial Settings"* (IEEE EuroS&P 2016) **Find at:** arXiv:1511.07528

What it does: JSMA (Jacobian Saliency Map Attack) — identifies the SMALL number of features with the BIGGEST impact on the model output using the Jacobian matrix. Perturbs ONLY those few features, but by a LARGE amount. Opposite of FGSM/PGD: few features changed a lot vs. all features changed a little.

Why it matters for our thesis: Our `native_sparse_attack()` in v5 Section 21 is based on this: perturb only 5% of features by the full epsilon budget. Our defended model maintains 92–95% accuracy against it — showing generalization across "attack shapes" (dense+small vs. sparse+large). BUT this high accuracy also fits the gradient masking explanation. Explain the nuance.

2.4 — MI-FGSM (Momentum Iterative FGSM)

Paper: Dong, Liao, Pang, Su, Zhu, Hu, Li — *"Boosting Adversarial Attacks with Momentum"* (CVPR 2018) **Find at:** arXiv:1710.06081

What it does: Adds a momentum term (running average of past

gradient directions) to the gradient attack. Like a rolling ball building speed — accumulates direction information across iterations, prevents oscillation, escapes flat gradient regions much better than plain PGD. Won the NIPS 2017 adversarial attacks competition.

Key formula: `g_{t+1} = mu * g_t + gradient / ||gradient||_1` `x_{t+1} = x_t + alpha * sign(g_{t+1})`

Why it matters for our thesis: `native_mifgsm()` in v5 Section 21 — a ZERO-DAY attack: model was NEVER trained against it. Our defended model achieves 92–95% accuracy against MI-FGSM. BUT: this high accuracy also fits the gradient masking explanation — if gradients are obfuscated, even a smarter gradient attack cannot find the right direction. Note this nuance explicitly: high accuracy against MI-FGSM does NOT rule out gradient masking.

SECTION 3 — Defenses: How do you make a model resist these attacks?

3.1 — Adversarial Training (Madry et al., already cited in 1.2)

Do NOT repeat the full citation. Write 2–3 sentences only: Briefly remind the reader that adversarial training (Madry et al., 2018) is our defense. Describe how Stage 1 (clean training on SHAP-robust features) + Stage 2 (adversarial training) combine in our pipeline. Reference Madry et al. (2018) as "(see Section 1.2 above)".

3.2 — TRADES

Paper: Zhang, Yu, Jiao, Xing, El Ghaoui, Jordan — *"Theoretically Principled Trade-off between Robustness and Accuracy"* (ICML 2019)

Find at: arXiv:1901.08573

What it does: Instead of training on adversarial EXAMPLES with their original labels (Madry), TRADES adds a KL-divergence regularization term to the loss:

```
Loss_TRADES = CrossEntropy(f(x_clean), y_true)
            + beta * KL_divergence( f(x_clean) || f(x_adv
```

The KL term penalizes the model when its output CHANGES between clean and adversarial input. Beta controls the clean-accuracy vs. robustness tradeoff. TRADES has theoretical guarantees connecting the loss to certified robustness bounds.

Why it matters for our thesis: We implement TRADES as an optional Stage 2 mode in v5 (`stage2_method="trades"`, `trades_beta=6.0` in CONFIG). Our comparison of TRADES masking gap vs. standard PGD masking gap is an **original contribution**. The motivation: TRADES explicitly regularizes the model to keep CONSISTENT predictions between clean and adversarial inputs — this naturally smooths gradients rather than obfuscating them, making it less prone to gradient masking than pure adversarial-example augmentation.

3.3 — Fast Adversarial Training (FGSM-RS)

Paper: Wong, Rice, Kolter — *"Fast is Better than Free: Revisiting Adversarial Training"* (ICLR 2020) **Find at:** arXiv:2001.03994

What it does: Shows that adversarial training does NOT need many PGD steps. A SINGLE FGSM step with a RANDOM starting point ("FGSM-RS") trains almost as robust a model as 20-step PGD training, at roughly 1/20th the compute cost. Key insight: the random start prevents "catastrophic overfitting" that makes naive single-step FGSM training fail. Without the random start, the model memorizes the perturbation direction and stops generalizing.

Why it matters for our thesis: This paper motivates our `adv_train_gradfree_ratio=0.20` parameter in v5 CONFIG. 20%

of adversarial examples per batch come from gradient-free random search instead of PGD. Explain the engineering tradeoff: More PGD steps = stronger training attacks = more compute per batch = slower training. Our v5 uses only 5 PGD steps during Stage 2 training (vs. 20 for evaluation) as a deliberate speed/strength tradeoff following the spirit of this paper.

3.4 — Multi-Perturbation Adversarial Training

Paper: Tramèr, Boneh — *"Adversarial Training and Robustness for Multiple Perturbations"* (NeurIPS 2019) **Find at:** arXiv:1904.13000

What it does: Training against ONE attack type (e.g., L∞ PGD) does NOT automatically give robustness against OTHER attack types (e.g., L2, spatial transforms). Robustness is specific to the threat model used during training. Proposes training against a MIXTURE of different threat models.

Why it matters for our thesis: This directly explains WHY v5 adds gradient-free random-search examples to the Stage 2 training mix. If we train ONLY on gradient-based PGD, the model learns to resist gradient-based attacks specifically — but may stay vulnerable to gradient-free (decision-based) attacks. This is the theoretical backing for our Priority 1a implementation. Also justifies our zero-day test suite design: testing across qualitatively different attack types (gradient-based, gradient-free, sparse, random) is the correct evaluation methodology.

SECTION 4 — Feature selection: why we pick some features and not all of them

Set this context before individual papers: CSE-CIC-IDS2018 has ~80 features. Not all are useful for classification, and not all are stable under adversarial attack. We use TWO stages of feature selection: Stage 1 (MI): reduces 80 → ~25 (paper/CUDA) or ~42 (v5 adaptive) Stage 2

(SHAP stability): reduces Stage 1 result → ~15 (paper/CUDA) or ~24 (v5 adaptive) Why this matters: (1) fewer features = smaller attack surface, (2) stable features = more robust model, (3) makes the model more interpretable.

4.1 — SHAP (SHapley Additive exPlanations)

Paper: Lundberg, Lee — *"A Unified Approach to Interpreting Model Predictions"* (NeurIPS 2017) **Find at:** arXiv:1705.07874

What it does: SHAP assigns each input feature a "contribution score" for every individual prediction, based on Shapley values from cooperative game theory. The mean absolute SHAP value across many predictions = overall feature importance. Unlike model weights, SHAP correctly handles correlated features and feature interactions.

Our two specific uses:

1. **Feature stability ranking:** Compute SHAP on both clean and PGD-adversarial inputs (baseline model). Stability = `1 − |SHAP_clean − SHAP_adv| / SHAP_clean` per feature. High stability = feature's importance does not change under attack = "robust feature."
2. **Feature subset for defended model:** Keep only the stable features for Stage 1 and Stage 2 training.

Key features identified by Holla et al. (mention in review): From their Section V.B and Figure 3: Protocol, Source Port, Destination Port, Flow Duration, Packet Length, Flow Bytes/s, Flow Packets/s, Forward Packet Length — most important AND most adversarially manipulated. "Protocol-based and statistical traffic attributes are particularly susceptible to adversarial manipulation."

Critical point to make — stability ≠ importance: A feature can be IMPORTANT but UNSTABLE (bad for defense) or LESS important but STABLE (good for defense). Our defense picks STABILITY, not raw importance.

Original finding from our v5: Paper uses fixed T=15. Our adaptive

90% threshold selects ~24 SHAP features. More features = larger attack surface = potentially weaker defense. This is one of our original contributions (Section 9 / Gap 2 of this document).

4.2 — Mutual Information for Feature Selection

Cite one of: OPTION A (textbook): Cover, Thomas — *"Elements of Information Theory"* (Wiley, any edition) OPTION B (paper): Guyon, Elisseeff — *"An Introduction to Variable and Feature Selection"* (JMLR 2003)

Explain in plain language: Mutual information between feature X and label Y = how much knowing X reduces uncertainty about Y. MI = 0: knowing this feature tells you nothing about whether traffic is an attack. High MI: knowing this feature strongly predicts the label.

Two approaches compared in our thesis:

- CUDA notebook (paper): Fixed T=25 by MI score
- v5 notebook (our approach): Adaptive — keep fewest features whose cumulative MI ≥ 90%

KEY ORIGINAL FINDING: On CSE-CIC-IDS2018, the adaptive 90% threshold selected ~**42 features** (of 44 candidates after variance filter) — almost all features. This means many features have similar MI scores; the 90% threshold is a WEAK filter on this dataset. The paper's fixed T=25 is a STRONGER filter. This is Gap 2 of our original contributions: we show adaptive cumulative-threshold feature selection performs WORSE than simple fixed-T on this dataset. This motivated our threshold sweep (v5 Section 26, 0.70/0.80/0.90) and elbow detection (v5 Section 25).

SECTION 5 — IS THE DEFENSE ACTUALLY WORKING? (Most important — do NOT rush this)

Set this context before the paper: A model can APPEAR robust without actually BEING robust. This happens when the defense accidentally **hides** or **distorts** the gradient used by attacks, instead of genuinely hardening the decision boundary. If gradients are hidden: gradient-based attacks cannot find the right direction → model LOOKS robust. A gradient-FREE attack ignores gradients completely → can still find and exploit the real boundary. This is called **gradient masking** or **gradient obfuscation**.

Our specific numbers showing gradient masking:

- Defended model vs. all gradient-based attacks (FGSM, PGD, DeepFool, MI-FGSM, Sparse): **92–95%** accuracy
- Defended model vs. Boundary-Walk (gradient-free): **34.5%** accuracy
- Gap = **59.15 percentage points**
- Our alert threshold = 15pp — this is nearly 4× the threshold
- Held consistently across seeds 42, 123, 2024 — NOT a random artifact

---

5.1 — Obfuscated Gradients Paper — THE SINGLE MOST IMPORTANT PAPER IN THIS ENTIRE DOCUMENT

Paper: Athalye, Carlini, Wagner — *"Obfuscated Gradients Give a False Sense of Security: Circumventing Defenses to Adversarial Examples"* (ICML 2018) **Find at:** arXiv:1802.00420

What it does: Analyzed 9 defenses published at ICLR 2018. Showed 7 of 9 were NOT actually robust — they worked by making gradient computation unreliable (obfuscated gradients) without genuinely hardening the decision boundary. A gradient-free attack could bypass all 7 of them immediately.

The 4 diagnostic signs of gradient masking — copy this list exactly into your review:

Sign	What it means	Our result

Sign 1	White-box gradient attack performs WORSE than black-box gradient-free	✓ PGD (white-box) ~5–8% SR; Boundary-Walk (black-box, gradient-free) 65.5% SR
Sign 2	Attack success rate INCREASES with more PGD iterations/restarts	✓ Our Table 10: SR increases from 20→200 iterations
Sign 3	Random perturbations are MORE effective than gradient-guided ones	✓ Random noise attack SR similar to structured PGD
Sign 4	Accuracy under gradient attacks is suspiciously near 100%	✓ 92–95% across all gradient attacks — unrealistically high for genuine robustness

Mitigations the paper recommends — we implement all three:

1. Include gradient-free attacks in TRAINING → our `adv_train_gradfree_ratio` (v5 Section 10)
2. Use TRADES regularization → our `stage2_method="trades"` (v5 Section 29)
3. Validate with adaptive attacks (more steps + restarts) → our Table 10 (v5 Section 28)

WHY THIS MATTERS MORE THAN ANY OTHER PAPER: Every single experiment we ADD in v5 beyond Holla et al. (Sections 25–31) exists because of this paper's diagnostic methodology. Cite Athalye et al. (2018) every time you mention: gradient masking, the 59.15pp gap, Boundary-Walk results, Table 10, TRADES, multi-seed stability.

SECTION 6 — The dataset and the paper we are reproducing

6.1 — CSE-CIC-IDS2018 Dataset

Paper: Sharafaldin, Lashkari, Ghorbani — *"Toward Generating a New Intrusion Detection Dataset and Intrusion Traffic Characterization"* (ICISSP 2018) **Find at:** Google Scholar: "CIC IDS 2018 Sharafaldin Ghorbani" | also: Canadian Institute for Cybersecurity website

What the dataset contains (use these specific facts):

- 10 days of network traffic, February–March 2018
- 7 attack categories: Brute Force (FTP, SSH), DoS/DDoS (Hulk, GoldenEye, Slowloris, LOIC HTTP, HOIC), Heartbleed, Web Attacks (XSS, SQL Injection, Brute Force), Infiltration, Botnet (ARES)
- ~80 flow features per record extracted by CICFlowMeter (inter-arrival times, packet lengths, flow durations, TCP flags, port numbers, protocol, byte/packet rates, etc.)
- Heavily class-imbalanced: benign traffic >> attack traffic

Our setup: Binary classification: all attack types → "attack" (1), normal → "benign" (0). All 10 days loaded (`day_patterns: None`). After variance threshold + MI selection: ~42 features (v5 adaptive) or 25 features (CUDA fixed-T).

Limitation acknowledged by Holla et al. (mention this): "Further validation on datasets such as UNSW-NB15 or NSL-KDD would provide stronger evidence of generalizability." We share this limitation.

6.2 — The Paper We Are Reproducing — Read This One Very Carefully

Paper: Holla, Polepalli, Sasikumar — *"Adversarial Threats to Cloud IDS: Robust Defense With Adversarial Training and Feature Selection"* (IEEE Access, 2025) **You already have the PDF** in docs/ folder.

Summary of method:

1. Dataset: CSE-CIC-IDS2018, binary classification
2. Feature selection: VarianceThreshold → MI ranking → top-T features (T not explicitly stated in paper) → SHAP stability → top-

T SHAP features (T not stated) — our CUDA uses T=25 MI, T=15 SHAP as estimates
3. Baseline: 3-layer MLP (widths not stated — we use 128→64→32), BatchNorm, Dropout=0.3, Adam lr=0.001, 50 epochs
4. Defense Stage 1: Retrain clean on SHAP-robust feature subset, 50 epochs
5. Defense Stage 2: Pre-generate FGSM+PGD+DeepFool examples (ART), mix with clean data, fine-tune 15 epochs
6. Evaluate: FGSM, PGD, DeepFool (eps=0.05), Square Attack

Exact claimed results — use these numbers in your review:

Table 4 — Accuracy, F1, Precision, Recall:

Condition	Accuracy	F1	Precision	Recall
Baseline (clean)	85%	83%	84%	82%
FGSM attack	60%	58%	61%	55%
PGD attack	55%	52%	54%	50%
DeepFool attack	50%	48%	49%	47%
With defense	88%	86%	87%	85%

Table 5 — Square Attack:

Model	Accuracy	F1	Attack SR
Baseline	63%	61%	37%
Defense	84%	83%	16%

Table 6 — Overhead:

Model	Train time	Infer time	Overhead
Baseline	3.0 hrs	10 ms/sample	—
Defense	3.75 hrs	11 ms/sample	~25%

Limitations the paper itself explicitly states (cite at least 3 in your review):

1. ~25% training overhead may not be feasible for resource-constrained environments
2. Adaptive adversaries can develop bypass techniques requiring continuous retraining
3. SHAP feature importance varies across attack types
4. Only CSE-CIC-IDS2018 evaluated — needs UNSW-NB15, NSL-KDD validation
5. Binary classification only — no granular multi-class detection
6. Functional integrity of adversarial perturbations not explicitly validated

Future work the paper explicitly lists (mention at least 2):

1. Federated Learning integration for decentralized, privacy-preserving cloud IDS
2. Hybrid defense: adversarial training + anomaly detection for zero-day attacks
3. Multi-class classification extension
4. Additional benchmark dataset evaluation

SECTION 7 — Why the dataset is hard to train on (class imbalance + SMOTE)

7.1 — SMOTE

Paper: Chawla, Bowyer, Hall, Kegelmeyer — *"SMOTE: Synthetic Minority Over-Sampling Technique"* (JAIR 2002) **Find at:** JMLR.org (free) | Google Scholar: "SMOTE Chawla 2002"

What it does: In a class-imbalanced dataset, naive oversampling (duplicating minority examples) causes memorization. SMOTE creates GENUINELY NEW synthetic minority examples by:

1. Take minority example x

2. Find its k nearest neighbors (also minority class)
3. Pick a random point on the LINE SEGMENT between x and a neighbor
4. That point is a new synthetic training example

The model sees new minority-class data it has never seen → better generalization.

Our specific use: CSE-CIC-IDS2018 is heavily imbalanced (benign >> attack traffic). Apply SMOTE AFTER train/test split, ONLY to training data. NEVER apply SMOTE to test data (would leak test information into training = data leakage = invalid results). `SMOTE(random_state=DATA_SEED)` — DATA_SEED=42 is FIXED across all runs so the balanced dataset is IDENTICAL regardless of which model seed we use.

Why it matters for our thesis: Without SMOTE, a model can achieve 90%+ accuracy just by predicting "benign" for everything — useless as IDS. F1 score (harmonic mean of precision and recall) is better than accuracy for imbalanced data because it penalizes ignoring the minority class. We report BOTH accuracy AND F1 in all tables.

SECTION 8 — Context: Deep learning for intrusion detection (background)

Write this to motivate the whole thesis. Use at least one paper from each sub-section.

8.1 — Survey of deep learning for IDS

Recommended: He, Kim, Asghar — *"Adversarial Machine Learning for Network Intrusion Detection Systems: A Comprehensive Survey"* (IEEE Communications Surveys & Tutorials, 2023) [This is Reference [4] in Holla et al. 2025 — directly connected]

What to write: Traditional signature-based IDS fails against new attack types. Deep learning IDS learns patterns from data without explicit programming. Advantage in cloud: high traffic volume and diverse attacks make hand-crafted signatures impractical. Vulnerability: the learned decision boundary can be exploited by adversarial inputs. Set up the tension: ML-based IDS is more powerful AND more vulnerable than signature-based IDS.

8.2 — Adversarial attacks specific to network IDS

Recommended: McCarthy, Ghadafi, Andriotis, Legg — *"Functionality Preserving Adversarial Machine Learning for Robust Classification in Cybersecurity and Intrusion Detection Domains: A Survey"* (Journal of Cybersecurity and Privacy, 2022) [This is Reference [12] in Holla et al. 2025]

What to write: Adversarial attacks on network IDS have SPECIFIC challenges vs. image classification:

1. Feature validity: cannot set packet length to negative or TCP flags to impossible values
2. Feature space is TABULAR, not images (why ART's SquareAttack is image-only — we wrote our own)
3. Attacker must generate valid network traffic, not just manipulated numbers The L∞ epsilon=0.05 constraint is an APPROXIMATION of the functional integrity constraint. Acknowledge this as a limitation of our evaluation — same limitation Holla et al. explicitly state.

SECTION 9 — Our original contributions vs. existing literature (gap analysis)

Write this LAST, after you have written all other sections. For each gap: (a) what exists, (b) what we add.

Gap 1: Gradient masking in tabular IDS classifiers

Athalye et al. (2018) documented gradient masking in image classifiers. There is limited work applying this diagnostic to tabular network traffic classifiers. **WE ADD:** First application of the Athalye diagnostic checklist to an adversarially-trained tabular IDS, finding a 59.15pp gradient-free vs. gradient-based accuracy gap, consistent across 3 seeds.

Gap 2: Feature selection sensitivity for adversarial robustness

Holla et al. (2025) use fixed T without sensitivity analysis. **WE ADD:** (a) Elbow-detection alternative to fixed-T and cumulative-threshold selection (v5 Section 25) (b) Threshold sweep at 0.70/0.80/0.90 showing feature count changes the masking gap (v5 Section 26) (c) Finding: adaptive 90% threshold selects ~42 of 44 features on this dataset — nearly no filtering — making fixed T=25 a more effective regularizer on this specific dataset.

Gap 3: On-the-fly vs. static adversarial training for tabular IDS

Holla et al. use static pre-generated adversarial training data (approximation of Madry). **WE ADD:** True on-the-fly Madry PGD training for a tabular IDS and comparison against static approach.

Gap 4: TRADES for tabular network intrusion detection

TRADES (Zhang et al., 2019) has been validated primarily in image classification. **WE ADD:** Application of TRADES to tabular network traffic classification and direct comparison of its gradient masking gap against standard Madry PGD training (v5 Section 29 — original contribution).

Gap 5: Multi-seed reproducibility of the gradient masking

finding

A single-run finding could be a one-off experimental artifact. **WE ADD:** Full repetition of the zero-day suite across 3 seeds (42, 123, 2024), reporting masking gap mean ± std to confirm the finding is structurally reproducible, not noise (v5 Section 31).

CHECKLIST — Do not submit until every box is ticked

- ☐ **1.1** FGSM — formula, 85%→60% paper number, mention 3 uses in our code
- ☐ **1.2** PGD — saddle-point concept, 85%→55% paper number, **explain on-the-fly vs. static** (most important, 6+ sentences)
- ☐ **1.3** DeepFool — minimal L2 concept, 85%→50% paper number, **explicitly mention cap_to_eps() fix**
- ☐ **1.4** C&W — cite for "stronger attack defeats defenses" principle and Table 10 motivation
- ☐ **Section 2 intro** — white-box vs. black-box definition written
- ☐ **2.1** Square Attack — custom tabular implementation, paper numbers (SR=37%/16%), our number (SR=75.3%)
- ☐ **2.2** Boundary Attack — explain it, **explicitly state our 34.5% vs. 92–95% numbers and 59.15pp gap**
- ☐ **2.3** JSMA / sparse — contrast dense vs. sparse attack philosophy
- ☐ **2.4** MI-FGSM — momentum idea, note 92–95% accuracy BUT explain the gradient masking nuance
- ☐ **3.1** Madry reference (2–3 sentences, describe our Stage 1+2 structure)
- ☐ **3.2** TRADES — write the loss formula, trades_beta=6.0, explain why it reduces masking
- ☐ **3.3** Fast adversarial training — FGSM-RS, link to our adv_train_gradfree_ratio
- ☐ **3.4** Multi-perturbation — one-attack training doesn't generalize, motivates our zero-day suite
- ☐ **4.1** SHAP — stability formula, list 8 key features from Holla et

al., stability ≠ importance
- ☐ **4.2** Mutual information — plain language explanation, compare fixed-T vs. adaptive, state 42-feature finding
- ☐ **5.1** Athalye obfuscated gradients — **all 4 signs with our specific numbers**, 3 mitigations we implement
- ☐ **6.1** CIC-IDS2018 — list all 7 attack categories, ~80 features, binary setup, class imbalance
- ☐ **6.2** Holla et al. 2025 — ALL numbers from Tables 4/5/6, at least 3 explicit limitations
- ☐ **7.1** SMOTE — synthetic (not duplicated) generation, DATA_SEED isolation, F1 vs accuracy motivation
- ☐ **8.1** IDS survey paper (motivation: why ML IDS matters in cloud)
- ☐ **8.2** Adversarial attacks on IDS paper (challenges of tabular feature space)
- ☐ **Section 9** — all 5 gaps written, each with: what exists + what we add
- ☐ All our numbers used correctly: 34.5%, 92–95%, 59.15pp, 42 features (MI), 24 features (SHAP), 75.3% SR (our Square Attack)
- ☐ Papers are in section order — do NOT rearrange

QUICK REFERENCE: Paper → Location in our code

Paper	Where in our code
Goodfellow 2015 (FGSM)	`FastGradientMethod` via ART — Sections 13/14 (both notebooks)
Madry 2018 (PGD)	`native_pgd()` for on-the-fly training (v5); `ProjectedGradientDescent` via ART for eval
Moosavi-Dezfooli 2016 (DeepFool)	`DeepFool` via ART + `cap_to_eps()` — Sections 13/16
Carlini & Wagner 2017	Not implemented — cited for methodology and adaptive atta motivation

Andriushchenko 2020 (Square)	`tabular_square_attack()` — custom, both notebooks, only
Brendel 2018 (Boundary Attack)	`native_boundary_walk()` — v5 Section 21, primary ma detector
Papernot 2016 (JSMA)	`native_sparse_attack()` — v5 Section 21
Dong 2018 (MI-FGSM)	`native_mifgsm()` — v5 Section 21
Zhang 2019 (TRADES)	`adversarial_train_model(stage2_method="trade` v5 Section 29
Wong 2020 (Fast adv. training)	Motivates `adv_train_gradfree_ratio` — v5 CONFIG
Tramèr 2019 (Multi-perturbation)	Motivates gradient-free training mix — v5 Section 10
Lundberg 2017 (SHAP)	`shap.Explainer` — Sections 15/16 (CUDA), Section 16 (
Athalye 2018 (Obfuscated gradients)	Section 22 (masking detection), Section 28 (Table 10), Sect (multi-seed)
Chawla 2002 (SMOTE)	`SMOTE(random_state=DATA_SEED)` — Section 9 (both notebooks)
Sharafaldin 2018 (IDS2018)	The entire `data/` folder — all experiments use this dataset
Holla et al. 2025 (base paper)	`adversarial_cloud_ids_defense_implement_cuda` reproduces this directly

Last updated from: Holla et al. (2025) PDF (full verbatim extraction including Tables 4/5/6, all section headings, all references, conclusion, limitations), CUDA notebook

(adversarial_cloud_ids_defense_implement_cuda.ipynb — original paper implementation), v5 notebook (adversarial_cloud_ids_defense_rocm_v5.ipynb — thesis extension), thesis_improvement_implementation_prompt.md, docs/update.md, v5update.md pipeline documentation.